

Trabajo Práctico: Patrón Strategy

Alumno: Santino Loray

Materia: Trabajo de Diploma

Universidad: Universidad Abierta Interamericana (UAI)

Repositorio GitHub: https://github.com/santi-programs/patron-strategy-csharp

---

Consignas

1. De acuerdo al Código de Ejemplo del Mundo Real en C# del Patrón Strategy que se muestra en el link de DoFactory, elaborar el Diagrama de Clases.

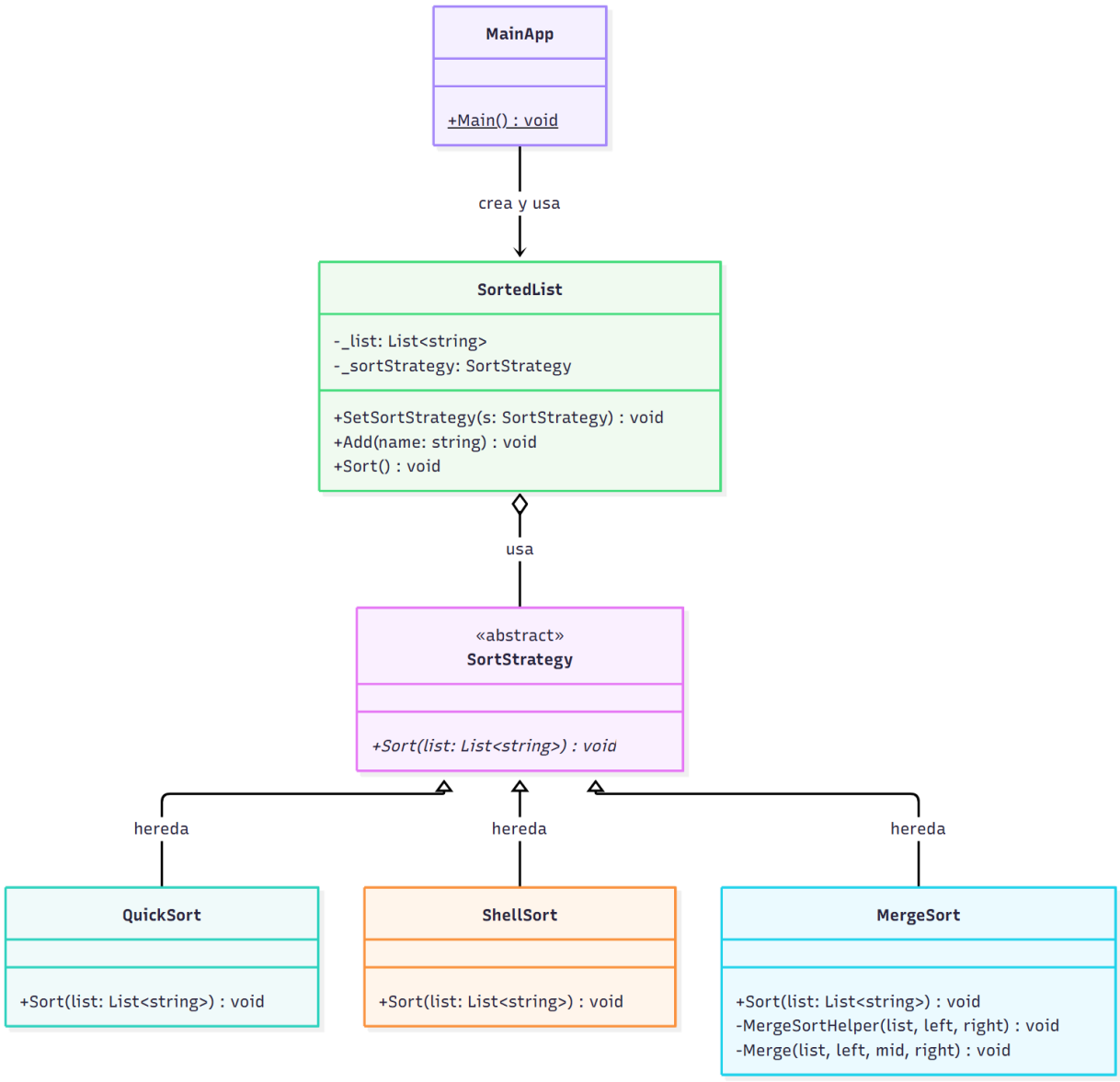
2. Abrir un proyecto nuevo de tipo Aplicación de Consola de C# en Visual Studio .Net y luego, copiar y pegar el código de ejemplo y realizar la ejecución paso a paso por instrucciones.

3. Colocar puntos de interrupción para inspeccionar el contenido de las variables a medida que se ejecuta el programa y evaluar el funcionamiento del código de acuerdo a la solución propuesta por el patrón.

4. Escribir en el foro de debate del tema todas las dudas o preguntas que consideren pertinentes.

Resolución

1. Diagrama de Clases



El diagrama de clases del Patrón Strategy aplicado al ejemplo del mundo real se compone de los siguientes participantes:

- **Strategy (SortStrategy):** Clase abstracta que declara la interfaz Sort() común a todos los algoritmos de ordenamiento.

- **ConcreteStrategy (QuickSort, ShellSort, MergeSort):** Implementan el algoritmo Sort() concreto. Cada una realiza el ordenamiento de manera diferente pero respetando la misma firma.
- **Context (SortedList):** Mantiene una referencia a SortStrategy. Delega el trabajo de ordenamiento al objeto estrategia actual. Permite cambiar la estrategia en tiempo de ejecución mediante SetSortStrategy().
- **Client (MainApp):** Instancia el contexto, carga los datos y selecciona dinámicamente la estrategia a utilizar.

2. Proyecto en Visual Studio .NET

Se creó un proyecto de tipo Aplicación de Consola en .NET 8 con el código del ejemplo del mundo real del Patrón Strategy basado en DoFactory. El proyecto se encuentra disponible en el repositorio GitHub indicado en la carátula.

El código implementa un sistema donde un objeto SortedList (contexto) puede cambiar dinámicamente su algoritmo de ordenamiento entre QuickSort, ShellSort y MergeSort sin modificar su propia lógica interna.

3. Ejecución Paso a Paso con Puntos de Interrupción

A continuación se detalla la ejecución paso a paso del programa, indicando los puntos de interrupción recomendados y el estado de las variables en cada momento:

PASO 1 — Creación del Context

Línea: SortedList studentRecords = new SortedList();

Variables en Breakpoint: _list = lista vacía, _sortStrategy = null

Análisis: Se instancia el objeto contexto SortedList. La lista interna está vacía y no hay ninguna estrategia asignada todavía.

PASO 2 — Carga de datos

Líneas: studentRecords.Add("Samual") hasta studentRecords.Add("Christopher")

Variables en Breakpoint: _list = ["Samual", "Jimmy", "Sandra", "Dave", "Christopher"]

Análisis: La lista interna acumula 5 nombres en el orden en que fueron agregados, sin ordenar.

PASO 3 — Estrategia QuickSort

Líneas: studentRecords.SetSortStrategy(new QuickSort()); seguido de studentRecords.Sort();

Variables en Breakpoint: _sortStrategy = instancia de QuickSort. Antes del sort: _list = ["Samual", "Jimmy", "Sandra", "Dave", "Christopher"]. Después del sort: _list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"].

Análisis: SetSortStrategy() asigna una instancia de QuickSort a _sortStrategy. Al llamar Sort(), se delega polimórficamente a QuickSort.Sort(list), que usa list.Sort() interno de C# para ordenar alfabéticamente. El foreach muestra cada elemento ordenado.

PASO 4 — Cambio de Estrategia: ShellSort

Líneas: studentRecords.SetSortStrategy(new ShellSort()); seguido de studentRecords.Sort();

Variables en Breakpoint: _sortStrategy = instancia de ShellSort. _list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"] (ya ordenada).

Análisis: Sin modificar la lista, simplemente se reemplaza la estrategia con ShellSort. El algoritmo Shell Sort itera con gaps decrecientes. La lista ya estaba ordenada, por lo que el resultado es el mismo. Esto demuestra la intercambiabilidad del patrón: el Context no sabe ni le importa qué algoritmo se usa internamente.

PASO 5 — Cambio de Estrategia: MergeSort

Líneas: studentRecords.SetSortStrategy(new MergeSort()); seguido de studentRecords.Sort();

Variables en Breakpoint: _sortStrategy = instancia de MergeSort. _list = ["Christopher", "Dave", "Jimmy", "Samual", "Sandra"].

Análisis: Se aplica el algoritmo Merge Sort recursivo (divide y conquista). Nuevamente, la lista ya ordenada produce el mismo output. Lo relevante es observar cómo el Context delega sin conocer el algoritmo interno. El dispatch polimórfico se puede verificar inspeccionando _sortStrategy.GetType() en el depurador.

Salida del programa en consola:

QuickSorted list: Christopher, Dave, Jimmy, Samual, Sandra

ShellSorted list: Christopher, Dave, Jimmy, Samual, Sandra

MergeSorted list: Christopher, Dave, Jimmy, Samual, Sandra

4. Preguntas para el Foro de Debate

Pregunta 1: ¿Por qué se usa abstract class en vez de interface para SortStrategy?

En C#, una clase abstracta puede contener lógica compartida entre estrategias (métodos con implementación parcial). Una interface obligaría a implementar todo en cada clase concreta. Sin embargo, si el contrato fuera solo un método puro, una interface sería igualmente válida. En este ejemplo, con un solo método abstracto, ambas opciones son equivalentes, pero interface sería más idiomático en C# moderno.

Pregunta 2: ¿Qué ventaja tiene Strategy sobre múltiples if/else o switch?

Con if/else, agregar un nuevo algoritmo requiere modificar el código existente, lo cual viola el Principio Abierto/Cerrado (OCP) de SOLID. Con Strategy, solo se agrega una nueva clase concreta sin tocar nada más. El código es más limpio, testeable de manera unitaria y fácilmente extensible.

Pregunta 3: ¿El Context (SortedList) conoce los detalles internos de cada algoritmo?

No. SortedList solo conoce la interfaz abstracta Sort(). Esto implementa el principio de inversión de dependencias (DIP): el módulo de alto nivel (SortedList) depende de la abstracción (SortStrategy), no de las implementaciones concretas. Esto permite agregar o modificar estrategias sin afectar al contexto.

Pregunta 4: ¿Se puede agregar BubbleSort sin modificar el código existente?

Sí. Solo se crea una nueva clase BubbleSort que herede de SortStrategy e implemente el método Sort(). Luego se usa: studentRecords.SetSortStrategy(new BubbleSort()). Esto cumple el Principio Abierto/Cerrado (OCP) de SOLID: abierto para extensión, cerrado para modificación.

Pregunta 5: ¿Qué diferencia hay entre Strategy y Template Method?

Strategy cambia el algoritmo completo mediante composición (referencia a objeto externo). Se puede cambiar en tiempo de ejecución. Template Method usa herencia: define el esqueleto del algoritmo en una clase base y deja que las subclases sobreescriban pasos específicos, quedando definido en tiempo de compilación. Strategy es más flexible cuando se necesita intercambiar algoritmos dinámicamente.

Repositorio GitHub público: https://github.com/santi-programs/patron-strategy-csharp

Fuente del código: https://www.dofactory.com/net/strategy-design-pattern